

CHAPTER 2

Theoretical Framework

2.1. Literature Review Methodology

A structured review with systematic elements grounds Chapter 2 and the artefact's design decisions in verified literature. Those elements are logged search strings, dated queries, explicit criteria, deduplication, snowballing and metadata verification. The review fits the purpose of mapping evidence across ten axes to ground design decisions, rather than answering one narrow question by exhaustive synthesis.

Table 2.1 lists the ten review axes with their topics and search strings. The planned primary databases are the Institute of Electrical and Electronics Engineers (IEEE) Xplore, the Association for Computing Machinery (ACM) Digital Library, Scopus and Web of Science. Included sources relate to at least one axis and are articles, studies, standards or official documentation in English. The criteria exclude duplicates, preprints without a peer-reviewed version, marketing material without technical substance and non-archival abstracts, posters or slide decks.

Table 2.1: Review axes and their database-neutral search strings.

Axis	Topic	Database-neutral search string
A	Edge computing and edge or IoT gateway architectures	("edge computing" OR "edge gateway" OR "IoT gateway") AND (architecture OR platform) AND (evaluation OR benchmark* OR "case study")
B	Digital twins: concepts, surveys and edge deployments	"digital twin*" AND (edge OR gateway OR "resource-constrained" OR embedded) AND (implementation OR evaluation OR survey)
C	Digital-twin platforms: Eclipse Ditto and alternatives	("Eclipse Ditto" OR "digital twin platform" OR "digital twin framework") AND (IoT OR edge)
D	Embedded Linux build systems and reproducible builds	("Yocto" OR "Buildroot" OR "embedded Linux" OR "reproducible build*") AND (image OR "build system" OR distribution)
E	Containers and virtualisation on ARM and constrained devices	(container* OR Docker OR "OCI") AND (ARM OR aarch64 OR "single-board" OR "edge device*" OR IoT) AND (overhead OR performance OR virtualization)
F	MQTT and IoT messaging: brokers, QoS and reliability	MQTT AND (broker* OR "quality of service" OR QoS OR reliability) AND (edge OR IoT) AND (comparison OR performance OR evaluation)

Axis	Topic	Database-neutral search string
G	W3C Web of Things and interoperability	("Web of Things" OR "Thing Description" OR "WoT") AND (interoperab* OR "digital twin" OR IoT)
H	Wearable telemetry and synthetic workload generation	(wearable* OR smartwatch OR "smart ring" OR "smart clothing" OR "e-textile*") AND (telemetry OR "data stream*" OR "data rate*" OR simulat* OR emulat*)
I	Performance evaluation and benchmarking methodology	(benchmark* OR "performance evaluation" OR "measurement methodology") AND (edge OR IoT OR cloud) AND ("confidence interval*" OR statistic* OR repetition* OR reproducib*)
J	C2DTA line and consumer-controlled digital twins	("digital twin*" AND (consumer OR "user-controlled" OR "data sovereignty")) OR "consumer-controlled digital twin"

2.2. Edge Computing and IoT Gateways

The *edge* covers any computing and network resource between data sources and cloud data centers, from a smartphone to a micro data center [2]. *Cloudlets*, also called micro data centers or fog nodes, hold only state cached from the cloud and run arbitrary code, typically inside a virtual machine or container[1]. Fog computing, a term introduced for this dispersed infrastructure[1], is like edge computing [2], but focuses more on the infrastructure side. Its motivation is the scalability of Internet of Things (IoT) infrastructure rather than the interactive performance of mobile applications [1].

In the fog model, a gateway device typically gathers data from all network sensors, manages cloud connectivity and performs initial processing, aggregation and analytics[8]. A conceptual edge operating system for a home gateway adds naming, service management and a data abstraction layer that removes noise and detects events[2]. The centralized design makes the gateway a single point of failure, a reliability drawback of fog computing[8].

A second reliability concern is operation through periods of lost connectivity. During outages, a cloudlet can perform the critical services of the cloud [1], and an industrial gateway can operate as an isolated device with its own decision-making [8]. Reconnection then calls for reconciliation of changes made offline [1], [9]. The hybrid irrigation system [9] lets users access a replicated edge instance during Internet outages and resolves schedule conflicts in favor of the edge version. None of the three articles reports a measured value from a disconnected or reconnection phase [1], [8], [9]. Although in [9] it announces tests across connected, offline and reconnection states, its reported results are average and peak throughput broken down by storage device and request interval. By contrast, the evaluation of this dissertation includes a disconnect and reconnect scenario for the local twin core among its six scenarios.

2.3. Digital Twins and Twin Platforms

Calling a system a digital twin does not settle how its data flows. The survey in [4] rates the link itself, defining the twin as the constant connection between a physical

object and its software representations, called logical objects. Strong connection is a constant, bidirectional link, whereas simple connection is unidirectional, not real time or interruptible[4]. The same survey reports a trend of distributing twin processing over edge, fog and cloud computing[4].

Eclipse Ditto is an open-source, domain-agnostic framework that represents each device as a Thing with attributes and features and separates reported, desired and current state[10]. Its documentation states that Ditto is not an end-to-end IoT platform and ships no software for devices or gateways[10]. Devices are integrated through a device connectivity layer such as an MQTT broker like Eclipse Mosquitto [10]. Ditto-managed connections to such layers and to other systems use MQTT, the Advanced Message Queuing Protocol (AMQP), Apache Kafka or HTTP [10].

The C2DTA tests run Ditto 3.0.0 on a server virtual machine and time each step of seven smartwatch lifecycle scenarios, mostly identity and ledger steps[6].

2.4. Embedded Linux and Containers on ARM64

The Yocto Project is an open-source collaboration for building custom Linux systems for embedded products, with Poky as its reference distribution [11]. Its metadata sits in *layers*, repositories of related build instructions and configuration, and BitBake schedules the build tasks [11]. Release 5.0, Scarthgap, has long-term support from April 2024 to April 2028[11]. Only the board support package layers are platform-specific, so swapping them retargets the same platform stack to another board [12]. Retargeting still implies a rebuild, a minimal Raspberry Pi image that stacks Raspberry Pi and application layers needs a separate compilation for each board whose processor differs [13]. This dissertation also builds two machine targets from one set of layers: a QEMU target for functional checks and an ARM64 virtual-machine target for the work.

Within the sources examined, evidence on build reproducibility comes from nixpkgs, a general-purpose package collection[14]. Across 709,816 rebuilds from 17 revisions, 99.68% to 99.95% of packages rebuild successfully, and 69% to 91% generate bitwise identical artefacts apart from a 2020 regression [14]. For embedded images, the Yocto documentation limits complete reproducibility to BitBake and OpenEmbedded-Core, and any added layer voids that guarantee [11].

Containers share the host's kernel and instruction set, so an image built for x86 does not run natively on an ARM device [15]. The same survey finds too few synthetic benchmarks for containerized ARM edge devices [15]. On an ARM64 Raspberry Pi 4, a benchmark of Docker, Podman and Singularity against native execution runs one workload at a time [16]. None of these measurements covers cooperating services in separate containers on native ARM64, such as a broker, twin platform, database and controller. This dissertation therefore verifies every container stack, runs that stack on the gateway's Yocto-built kernel and keeps the configurations in its versioned evidence package.

2.5. MQTT and the Web of Things

MQTT Version 5.0 is a Client Server publish/subscribe messaging transport protocol, used for Machine-to-Machine communication and Internet of Things contexts where is required a small code footprint and light weight implementation [17]. Three quality-of-service (QoS) levels, at most once delivery, at least once delivery and exactly once delivery, set the guarantee between one sender and one receiver [17].

A Web of Things (WoT) *Thing Description* describes a physical or virtual entity through metadata, properties, actions, events and protocol bindings [18]. One description can bind several protocols and typically uses JavaScript Object Notation (JSON), which allows JSON for Linked Data (JSON-LD) processing [18]. The C2DTA implementation uses MQTT for sensor-data transmission and incorporates WoT-based twin definitions into the twinning workflow. [6].

2.6. Wearable Telemetry and Synthetic Workloads

The survey in [3] sorts wearables into Accessories, such as smartwatches and smart rings, E-Textiles, such as smart clothing, and E-Patches. Surveyed smartwatches normally pair with a smartphone, predominantly over Bluetooth or Bluetooth Low Energy (BLE), and one smart item of clothing relays data through a hub module [3]. Several wearables per person may need scheduling against interference or data collision at the aggregating smartphone or cloud server [3].

In one of the studies, where used three wearables, at 10 Hz each, to measure three inertial measurement units in a rehabilitation system report three-axis orientation, acceleration and velocity [19]. Another two substance-use studies process electrodermal activity, skin temperature and three-axis movement as *data streams*, unbounded sequences of observations [20] , [21]. Their rates are 30 samples per second per patient in one [20] and a configurable 2 Hz to 32 Hz in the other [21]. These rates are study-specific, not general properties of wearables. Neither substance-use study generates synthetic streams, and their future work names emulation and simulated data sets respectively [20], [21] .

The framework in [22] emulates a wound-therapy wearable in hardware to prototype its software before manufacture. That software combines timer-driven periodic tasks with event-driven BLE input from the in-wound sensors [22]. A *synthetic workload* replaces device data with generated messages, without hardware emulation. The dissertation's simulator uses a nominal smartwatch message rate of 1.0 message/s, consistent with the reference study. It adds smart-ring and smart-clothing profiles configured at 0.2 and 10.0 messages/s, respectively, as experimental workload parameters.

2.7. Consumer-Controlled Digital Twins

The *personal data ecosystem* governs the collection and use of personal data involving consumers and manufacturers, and [6] describes it as unbalanced in favour of manufacturers. The C2DTA moves the smart-device twin from the manufacturer's cloud to the consumer's edge and builds on blockchain and self-sovereign identity (SSI) [6]. Decentralised identifiers

(DIDs), verifiable credentials and DIDComm provide its SSI functions [6]. A DID is a globally unique, persistent identifier that does not require a centralized registration authority [23]. The Verifiable Credentials Data Model defines tamper-evident credentials that issuers, holders and verifiers exchange [24], and DIDComm Messaging provide a secure and private communication methodology[25].

The C2DTA edge gateway is a DIDComm connectivity hub, hosts the twin platform, interfaces with the ecosystem and identity ledgers and pushes historical twin data to decentralized storage [6]. The article calls this gateway a single point of failure whose primary role is to safeguard data integrity and privacy [6]. Hyperledger Aries agents, Hyperledger Indy, Hyperledger Fabric and the InterPlanetary File System (IPFS) implement the agent, ledger and storage functions [6]. Sensor data reach Eclipse Ditto through Eclipse Mosquitto using MQTT, which C2DTA chooses instead of DIDComm to optimize performance[6].

The evaluation tests seven of the eight scenarios in a smartwatch lifecycle, across 88 steps [6]. Docker runs in a virtual machine on a server with 32 CPUs, 16 GB of memory and unstated processor architecture. One Python simulator publishes smartwatch data at 1 Hz. Most timed steps are identity and ledger operations, and the latency that ledger writes, and cryptographic operations add does not exceed two seconds. The article reports no throughput, processor-usage, memory-usage or repetition data, states that test scale limits the evaluation and names testing with ARM cloud infrastructure as future work [6].

This dissertation evaluates an integrated gateway hosting the local twin core that C2DTA places at the consumer's edge, with SSI, DIDComm, ledgers and IPFS as reference-architecture context.

2.8. Performance Benchmarking at the Edge

Performance benchmarking stresses a system under test while observing its responses through quality metrics [26]. In the literature surveyed up to 2020, explicit benchmarking research, which builds a benchmark or toolchain, rarely considers orchestrators, service models or schedulers[26]. Most benchmarks deploy applications without virtualisation and so capture single-application performance, not edge environments where concurrent users share resources [26].

The following practices come mostly from studies without ARM edge hardware. An edge benchmarking framework separates its load generator from the service under test and emulates network conditions[27]. Replication differs, from 30 repetitions of each experiment on x86 cloud instances [27] to single-seed point estimates in a federated-learning benchmark[28]. That benchmark still writes each run to a self-contained folder of configuration, logs and per-round metrics [28]. A platform concept for embedded devices proposes cleaning all test elements so that each test starts under the same conditions [29]. Per-container monitoring is the exception: an example of a benchmark on a Raspberry Pi 4 monitors each container and calculates mean and standard deviation[16].

The procedure of this dissertation makes the run its statistical unit and measures processor and memory use per container. Like the self-contained folder, a versioned evidence package keeps the code, configurations, logs, data, checksums and analysis script. The work runs on a native ARM64 virtual machine that boots the Yocto-built image, with the load generator outside that measured machine.

2.9. Related Work and Research Gap

Table 2.2: Related systems compared with the planned prototype of this dissertation.

System	Twin platform	Hardware	Workload	Metrics reported
C2DTA [6]	Eclipse Ditto 3.0.0	Virtual machine on a server with 32 CPUs and 16 GB of RAM · architecture not stated	One simulated smartwatch at 1 Hz	Per-step response times · seven life-cycle scenarios
OpenTwins [30]	Eclipse Ditto · version not stated	Five-node Kubernetes cluster · architecture not stated	Data of 27 petrochemical sensors · simulated clients	Latency · throughput · recovery time · data loss
Vertical-farm twin [31]	Eclipse Ditto · version not stated	Raspberry Pi collector · Ditto host not stated	Greenhouse temperature · humidity · CO2 in 2021	None · verified by visualisation only
Modular edge twin [32]	Java edge twin · Eclipse Ditto as baseline	Intel Core i7 edge nodes	Emulated smart objects · 5 to 100 messages/s	End-to-end delay · startup time · processor and heap use
High-availability edge node [33]	None · Mosquitto broker and Python pipeline	Raspberry Pi 5 primary and standby pair	Environmental sensors · 10 Hz stream	Failover and recovery time · message loss · throughput · utilisation
Hybrid edge–cloud system [9]	None · Django edge and cloud instances	Orange Pi 3B edge server	Irrigation schedule requests · 50 to 250 ms intervals	Average and maximum throughput per storage device
This dissertation · planned	Eclipse Ditto on the Yocto-built image	Native ARM64 virtual machine booting that image · non-burstable	Three simulated wearable types · to specify messages/s nominal	Correctness · latency · throughput · saturation · per-container resources

Eclipse Ditto recurs in Table 2.2 as the twin layer of three related systems and as the comparison baseline of the modular edge twin [6], [30], [31], [32]. These twin platforms

run on Intel Core i7 edge nodes [32] or on hosts of unstated processor architecture [6], [30], [31]. Single-board computers serve as a sensor collector or as edge nodes without a twin platform [9], [31],[33]. The only simulated wearable device among these workloads is the 1 Hz smartwatch of the C2DTA [6]. An ARM64 container benchmark compares Docker, Podman and Singularity with native execution on a Raspberry Pi 4, one application at a time [16].

Within the sources examined, no evaluation combines a C2DTA-style local twin core on native ARM64 with concurrent telemetry from several wearable types. That combination is absent from the Intel-node delay results. The Raspberry Pi 4 benchmark runs no multi-container twin stack either. The reviewed sources do not report whether a Ditto-based digital twin platform running on ARM64 correctly updates twin states under concurrent telemetry streams or reliably handles telemetry interruptions and invalid payloads. They also do not identify the workload at which the platform saturates or quantify the resource consumption of individual containers[6],[9], [16], [30], [31], [32], [33]. The evaluation of this dissertation is designed to supply those four quantities, together with the latency and sustainable throughput of the integrated gateway. RQ1 asks how to build, deploy and redeploy a versioned Yocto-based ARM64 gateway image in an ARM64 virtual machine to host the local digital-twin core of C2DTA. RQ2 examines how correctly and reliably the integrated gateway turns concurrent synthetic telemetry from three simulated wearable types into twin state, also under dropout and invalid payloads. On the selected non-burstable ARM64 virtual machine, RQ3 targets the latency, sustainable-throughput, saturation and per-container resource trade-offs that characterize the integrated gateway under controlled loads.

References

- [1] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer (Long. Beach. Calif.)*, vol. 50, no. 1, pp. 30–39, Jan. 2017, doi: 10.1109/MC.2017.9.
- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.
- [3] S. Seneviratne *et al.*, “A Survey of Wearable Devices and Challenges,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 4, pp. 2573–2620, 2017, doi: 10.1109/COMST.2017.2731979.
- [4] R. Minerva, G. M. Lee, and N. Crespi, “Digital Twin in the IoT Context: A Survey on Technical Features, Scenarios, and Architectural Models,” *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1785–1824, Oct. 2020, doi: 10.1109/JPROC.2020.2998530.
- [5] A. Fuller, Z. Fan, C. Day, and C. Barlow, “Digital Twin: Enabling Technologies, Challenges and Open Research,” *IEEE Access*, vol. 8, pp. 108952–108971, 2020, doi: 10.1109/ACCESS.2020.2998358.
- [6] F. Pinto, C. Ferreira da Silva, S. Moro, and P. Aquino, “Consumer-controlled digital twin architecture: How blockchain technology gives consumers control over their smart devices’ digital twins and data,” *Blockchain: Research and Applications*, vol. 7, no. 3, p. 100342, Jun. 2026, doi: 10.1016/j.bcr.2025.100342.
- [7] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, Dec. 2007, doi: 10.2753/MIS0742-1222240302.
- [8] R. Dobrescu, S. Mocanu, O. Chenaru, M. Nicolae, and G. Florea, “Versatile edge gateway for improving manufacturing supply chain management via collaborative networks,” *Int. J. Comput. Integr. Manuf.*, vol. 34, no. 4, pp. 407–421, Apr. 2021, doi: 10.1080/0951192X.2021.1879401.
- [9] L. F. A. da Silva, P. R. da Silva Amalfi Costa, and C. Schepke, “Performance and Resilience Evaluation of a Hybrid Edge–Cloud IoT System Across Connected, Offline, and Reconnection Scenarios,” *Lecture Notes in Computer Science*, pp. 651–668, 2027, doi: 10.1007/978-3-032-30488-9_40.
- [10] Eclipse Foundation, “Eclipse Ditto: open source framework for digital twins in the IoT,” 2026. [Online]. Available: <https://eclipse.dev/ditto/>.
- [11] Yocto Project, “Yocto Project Documentation (Scarthgap, 5.0 LTS),” 2024. [Online]. Available: <https://docs.yoctoproject.org/scarthgap/>.

- [12] M. Ozcelikors and A. Gumuskavak, "Platform-independent Infotainment and Digital Cluster Development using Yocto Project," in *2020 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*, IEEE, Jun. 2020, pp. 1–5. doi: 10.1109/ICECCE49384.2020.9179288.
- [13] A. Biswas, D. Biswas, S. S. Chauhan, and A. Borwankar, "Smart home equipment control system with raspberry Pi and Yocto," *Proceedings of the World Conference on Smart Trends in Systems, Security and Sustainability, WS4 2020*, pp. 553–558, Jul. 2020, doi: 10.1109/WORLDS450073.2020.9210376.
- [14] J. Malka, S. Zacchiroli, and T. Zimmermann, "Does Functional Package Management Enable Reproducible Builds at Scale? Yes.," *Proceedings - 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, MSR 2025*, pp. 775–787, 2025, doi: 10.1109/MSR66628.2025.00115.
- [15] S. Kaiser, M. S. Haq, A. S. Tosun, and T. Korkmaz, "Container Technologies for ARM Architecture: A Comprehensive Survey of the State-of-the-Art," *IEEE Access*, vol. 10, pp. 84853–84881, 2022, doi: 10.1109/ACCESS.2022.3197151.
- [16] S. Kaiser, A. S. Tosun, and T. Korkmaz, "Benchmarking Container Technologies on ARM-Based Edge Devices," *IEEE Access*, vol. 11, pp. 107331–107347, 2023, doi: 10.1109/ACCESS.2023.3321274.
- [17] OASIS, "MQTT Version 5.0," 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>.
- [18] M. Lagally and M. McCool, "IoT Interoperability with W3C Web of Things," *Proceedings - IEEE Consumer Communications and Networking Conference, CCNC*, 2022, doi: 10.1109/CCNC49033.2022.9700546.
- [19] A. Candelieri, W. Zhang, E. Messina, and F. Archetti, "Automated Rehabilitation Exercises Assessment in Wearable Sensor Data Streams," *Proceedings - 2018 IEEE International Conference on Big Data, Big Data 2018*, pp. 5302–5304, Jul. 2018, doi: 10.1109/BIGDATA.2018.8621958.
- [20] J. Wang, H. Fang, S. Carreiro, H. Wang, and E. Boyer, "A new mining method to detect real time substance use events from wearable biosensor data stream," *2017 International Conference on Computing, Networking and Communications, ICCNC 2017*, pp. 465–470, Mar. 2017, doi: 10.1109/ICCNC.2017.7876173.
- [21] J. Rumbut, H. Fang, E. W. Boyer, and H. Wang, "Short: Precision Polysubstance Use Episode Detection in Wearable Biosensor Data Streams," *Proceedings - 2023 IEEE/ACM International Conference on Connected Health: Applications, Systems and Engineering Technologies, CHASE 2023*, pp. 163–167, 2023, doi: 10.1145/3580252.3586989.
- [22] Vasileios Tsoutsouras, Sotirios Xydis, and Dimitrios Soudris, "A HW/SW framework emulating wearable devices for remote wound monitoring and management | IEEE Conference Publication | IEEE Xplore," *2014 4th International Conference on Wireless Mobile Communication and Healthcare - Transforming Healthcare Through*

- Innovations in Mobile and Wireless Technologies (MOBIHEALTH)*, Nov. 2014, doi: 10.4108/Mobihealth33544.2014.7015988.
- [23] World Wide Web Consortium (W3C), “Decentralized Identifiers (DIDs) v1.0,” 2022. [Online]. Available: <https://www.w3.org/TR/did-core/>.
 - [24] World Wide Web Consortium (W3C), “Verifiable Credentials Data Model v2.0,” 2025. [Online]. Available: <https://www.w3.org/TR/vc-data-model-2.0/>.
 - [25] Decentralized Identity Foundation (DIF), “DIDComm Messaging Specification v2.1,” 2023. [Online]. Available: <https://identity.foundation/didcomm-messaging/spec/v2.1/>.
 - [26] B. Varghese *et al.*, “A Survey on Edge Performance Benchmarking,” *ACM Comput. Surv.*, vol. 54, no. 3, pp. 1–33, Apr. 2022, doi: 10.1145/3444692.
 - [27] O. Naman, H. Qadi, M. Karsten, and S. Al-Kiswany, “MECBench: A Framework for Benchmarking Multi-Access Edge Computing Platforms,” *Proceedings - IEEE International Conference on Edge Computing*, vol. 2023-July, pp. 85–95, 2023, doi: 10.1109/EDGE60047.2023.00024.
 - [28] M. V. S. Silva, C. A. Astudillo, and L. F. Bittencourt, “Federated Urban Flow: A Model-Centric Benchmark for Reproducible Evaluation of Deep Learning in Federated Traffic Forecasting,” *2025 3rd International Conference on Federated Learning Technologies and Applications, FLTA 2025*, pp. 499–505, 2025, doi: 10.1109/FLTA67013.2025.11336506.
 - [29] A. Garcia-Perez, R. Miñón, A. I. Torre-Bastida, J. Diaz-de-Arcaya, and E. Zulueta-Guerrero, “Conceptualising a Benchmarking Platform for Embedded Devices,” in *2024 9th International Conference on Smart and Sustainable Technologies (SpliTech)*, IEEE, Jun. 2024, pp. 1–4. doi: 10.23919/SpliTech61897.2024.10612407.
 - [30] J. Robles, C. Martín, and M. Díaz, “OpenTwins: An open-source framework for the development of next-gen compositional digital twins,” *Comput. Ind.*, vol. 152, p. 104007, Nov. 2023, doi: 10.1016/j.compind.2023.104007.
 - [31] T. H. Ko, H. M. Lee, D. H. Noh, C. Juhwan, and S. W. Byun, “Design and Implementation of a Digital Twin Platform in Vertical Farming Systems,” *International Conference on Ubiquitous and Future Networks, ICUFN*, vol. 2022-July, pp. 366–368, 2022, doi: 10.1109/ICUFN55119.2022.9829694.
 - [32] M. Picone, M. Mamei, and F. Zambonelli, “A Flexible and Modular Architecture for Edge Digital Twin: Implementation and Evaluation,” *ACM Transactions on Internet of Things*, vol. 4, no. 1, pp. 1–32, Feb. 2023, doi: 10.1145/3573206.
 - [33] M. Neagu, C. M. Serban, A. Hangan, and G. Sebestyen, “Digital Twins at the Edge: A High-Availability Framework for Resilient Data Processing in IoT Sensor Networks,” *Future Internet*, vol. 18, no. 3, p. 137, Mar. 2026, doi: 10.3390/fi18030137.